

# Events Attendance Monitoring System — Technical Plan

---

**Project:** Events Attendance Monitoring System for Students with RFID Integration — University of San Agustin **Prepared for:** Center for Information and Communications Support Service (CICSS) **Stack:** React (Web Admin) · React Native (Mobile RFID Capture) · .NET (Backend API) **Document version:** 1.0 — 2026-06-22

---

## Table of Contents

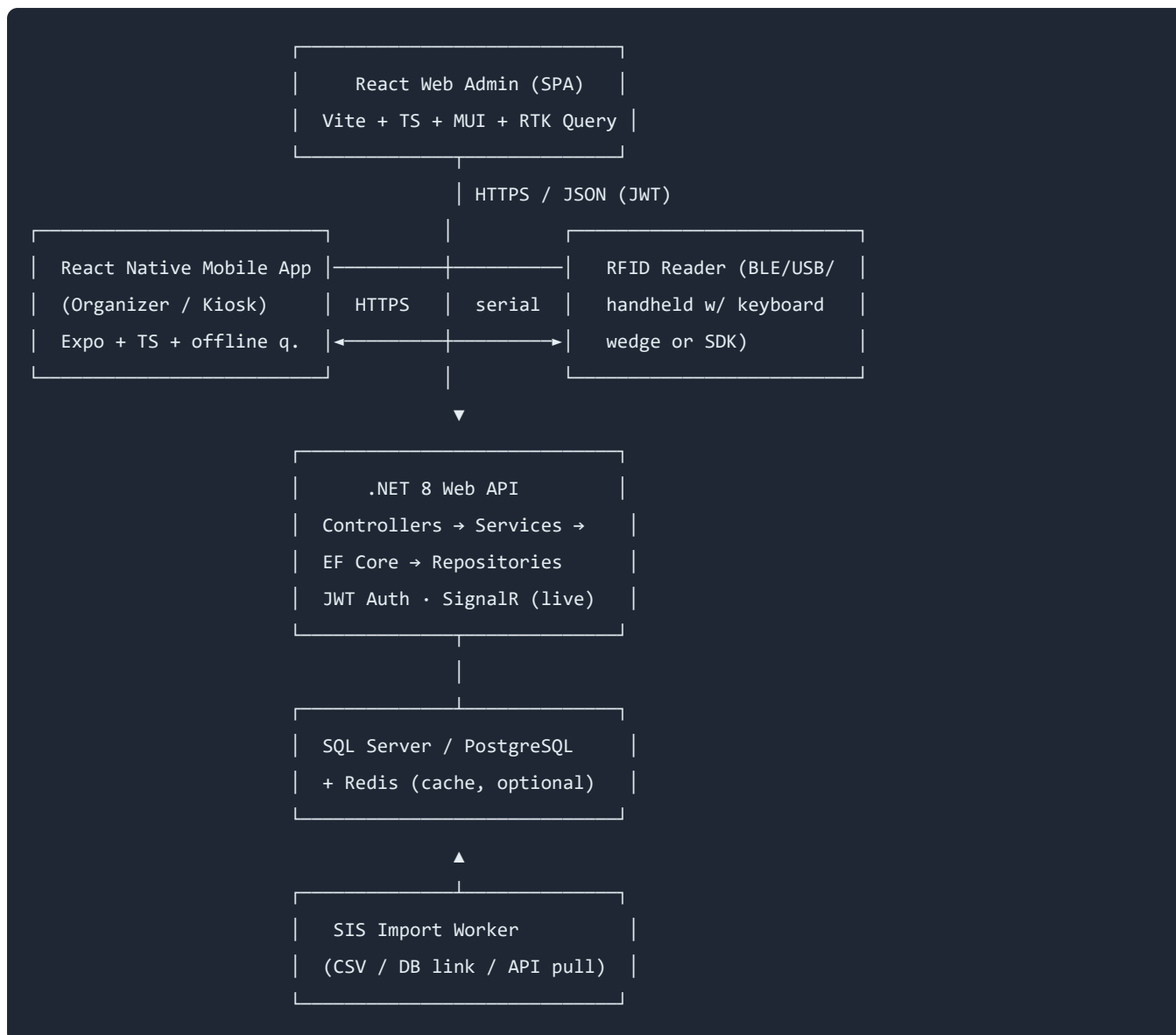
---

1. Architecture Overview
  2. Technology Stack
  3. Solution / Project Structure
  4. Database Design (Schema + Column Properties)
  5. Backend Modules (.NET)
  6. API Specification
  7. Frontend — Web Admin (React)
  8. Mobile App — RFID Capture (React Native)
  9. RFID Integration Design
  10. SIS Data Migration
  11. Authentication, Authorization & RBAC
  12. Reporting Module
  13. Deployment (SaaS & On-Premise)
  14. Cross-Cutting Concerns
  15. Implementation Roadmap
- 

## 1. Architecture Overview

---

A three-tier, API-centric architecture. A single .NET Web API is the source of truth; the React web admin and the React Native mobile app both consume the same REST/JSON API. RFID taps are captured on the mobile device (or a fixed kiosk running the same app) and posted to the API; reporting and management happen on the web.



**Key flows - Attendance capture:** Mobile reads RFID UID → resolves to student → POST

`/api/attendance/tap` → API validates against event window → persists → broadcasts via SignalR to web

dashboards in real time. - **Offline resilience:** Mobile queues taps locally (SQLite) when offline and syncs on

reconnect; the API de-duplicates by `(EventId, StudentId, deviceTapId)`. - **Reporting:** Web admin queries

aggregate endpoints; exports rendered server-side (PDF/CSV).

## 2. Technology Stack

| Layer             | Choice                                                        | Notes                                     |
|-------------------|---------------------------------------------------------------|-------------------------------------------|
| Backend framework | <b>.NET 8</b> (ASP.NET Core Web API)                          | LTS; minimal-hosting + controllers        |
| ORM               | <b>Entity Framework Core 8</b>                                | Code-first migrations                     |
| Database          | <b>SQL Server 2022</b> (default) / <b>PostgreSQL 16</b> (alt) | Provider-agnostic via EF Core             |
| Auth              | <b>JWT (access + refresh)</b> + ASP.NET Identity              | Role + permission claims                  |
| Real-time         | <b>SignalR</b>                                                | Live attendance dashboard                 |
| Caching           | <b>Redis</b> (optional)                                       | Session/RFID-map cache, rate limiting     |
| Background jobs   | <b>Hangfire</b>                                               | SIS imports, report generation, retention |
| API docs          | <b>Swagger / OpenAPI (Swashbuckle)</b>                        | Contract source                           |
| Logging           | <b>Serilog</b> → file + Seq/console                           | Structured logs                           |
| Validation        | <b>FluentValidation</b>                                       | Request DTO validation                    |
| Mapping           | <b>Mapster</b> or AutoMapper                                  | Entity ↔ DTO                              |
| Web frontend      | <b>React 18 + TypeScript + Vite</b>                           | SPA                                       |
| Web UI kit        | <b>MUI v5 + Recharts</b>                                      | Tables, forms, charts                     |
| Web state/data    | <b>Redux Toolkit + RTK Query</b>                              | Caching, invalidation                     |
| Web routing       | <b>React Router v6</b>                                        | Role-guarded routes                       |
| Web forms         | <b>React Hook Form + Zod</b>                                  | Typed validation                          |
| Mobile            | <b>React Native (Expo) + TypeScript</b>                       | Android-first (RFID handhelds)            |
| Mobile data       | <b>TanStack Query + expo-sqlite</b>                           | Offline queue + sync                      |
| Mobile nav        | <b>React Navigation</b>                                       | Stack + tabs                              |
| PDF/CSV           | <b>QuestPDF + CsvHelper</b> (server-side)                     | Report exports                            |
| CI/CD             | <b>GitHub Actions</b> / Azure DevOps                          | Build, test, deploy                       |
| Containerization  | <b>Docker + docker-compose</b>                                | On-prem & SaaS parity                     |

### 3. Solution / Project Structure

---

```
EAMS/
├─ backend/
│   └─ EAMS.sln
│       ├─ EAMS.Api                # Controllers, middleware, SignalR hubs, DI
│       ├─ EAMS.Application        # Services, DTOs, validators, interfaces
│       ├─ EAMS.Domain             # Entities, enums, domain rules
│       ├─ EAMS.Infrastructure     # EF Core DbContext, repositories, migrations, SIS, PDF
│       ├─ EAMS.Worker             # Hangfire jobs (SIS sync, reports, retention)
│       └─ EAMS.Tests              # Unit + integration tests
│
├─ web-admin/                     # React + Vite
│   └─ src/
│       ├─ api/                   # RTK Query slices (one per module)
│       ├─ app/                   # store, router, theme
│       ├─ features/              # students, events, attendance, users, reports...
│       ├─ components/           # shared UI (DataTable, FormField, Guard)
│       ├─ hooks/  utils/  types/
│       └─ layouts/
│
└─ mobile-rfid/                  # React Native (Expo)
    └─ src/
        ├─ api/                   # query client + endpoints
        ├─ rfid/                  # reader adapters (BLE / wedge / SDK)
        ├─ offline/              # SQLite queue + sync engine
        ├─ screens/              # Login, EventPicker, ScanScreen, Roster, Settings
        ├─ components/  hooks/  navigation/
        └─ store/
```

**Backend layering rule:** Api → Application → Domain; Infrastructure implements Application / Domain interfaces (Clean Architecture / dependency inversion). No EF types leak into the API surface — controllers speak DTOs only.

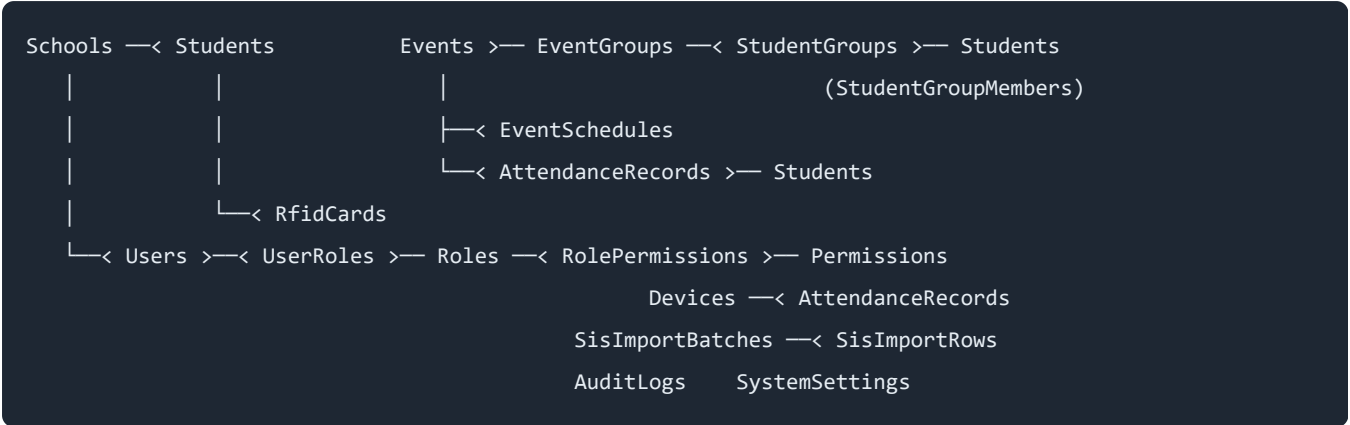
---

### 4. Database Design

---

Conventions: every table has Id (GUID, PK), CreatedAt, CreatedBy, UpdatedAt, UpdatedBy, and IsDeleted (soft delete) unless noted. Timestamps are UTC datetime2. GUIDs avoid SIS-ID collisions across migrations and are safe for distributed/offline generation.

## 4.1 ERD (logical)



## 4.2 Schools

| Column                | Type             | Constraints                     | Description          |
|-----------------------|------------------|---------------------------------|----------------------|
| Id                    | uniqueidentifier | PK                              | School identifier    |
| Name                  | nvarchar(200)    | NOT NULL                        | School / campus name |
| Code                  | nvarchar(50)     | NOT NULL, UNIQUE                | Short code           |
| Address               | nvarchar(500)    | NULL                            | Physical address     |
| ContactEmail          | nvarchar(256)    | NULL                            | Admin contact        |
| LogoUrl               | nvarchar(1000)   | NULL                            | For reports/branding |
| TimeZone              | nvarchar(64)     | NOT NULL, default 'Asia/Manila' | Local TZ for display |
| IsActive              | bit              | NOT NULL, default 1             | Soft enable          |
| CreatedAt / UpdatedAt | datetime2        | NOT NULL                        | Audit                |

## 4.3 Students

| Column                | Type             | Constraints                              | Description               |
|-----------------------|------------------|------------------------------------------|---------------------------|
| Id                    | uniqueidentifier | PK                                       | Internal id               |
| SchoolId              | uniqueidentifier | FK→Schools, NOT NULL                     | Owning school             |
| StudentNumber         | nvarchar(50)     | NOT NULL, UNIQUE(SchoolId,StudentNumber) | SIS student ID            |
| FirstName             | nvarchar(100)    | NOT NULL                                 |                           |
| MiddleName            | nvarchar(100)    | NULL                                     |                           |
| LastName              | nvarchar(100)    | NOT NULL                                 |                           |
| Email                 | nvarchar(256)    | NULL                                     |                           |
| Course                | nvarchar(150)    | NULL                                     | Program/course            |
| YearLevel             | nvarchar(50)     | NULL                                     | e.g. "1st Year"           |
| Section               | nvarchar(50)     | NULL                                     |                           |
| Gender                | nvarchar(20)     | NULL                                     |                           |
| PhotoUrl              | nvarchar(1000)   | NULL                                     | Optional ID photo         |
| Status                | nvarchar(20)     | NOT NULL, default 'Active'               | Active/Inactive/Graduated |
| SisExternalId         | nvarchar(100)    | NULL, INDEX                              | Original SIS PK for sync  |
| LastSyncedAt          | datetime2        | NULL                                     | Last SIS sync             |
| CreatedAt / UpdatedAt | datetime2        | NOT NULL                                 |                           |
| IsDeleted             | bit              | NOT NULL, default 0                      | Soft delete               |

**Indexes:** `IX_Students_SchoolId_StudentNumber` (unique), `IX_Students_SisExternalId` , full-text/like index on ( `FirstName` , `LastName` , `StudentNumber` ) for search.

## 4.4 RfidCards

| Column                | Type             | Constraints           | Description                     |
|-----------------------|------------------|-----------------------|---------------------------------|
| Id                    | uniqueidentifier | PK                    |                                 |
| StudentId             | uniqueidentifier | FK→Students, NOT NULL | Owner                           |
| CardUid               | nvarchar(128)    | NOT NULL, UNIQUE      | RFID tag UID (hex)              |
| Label                 | nvarchar(100)    | NULL                  | e.g. "Primary ID"               |
| IsActive              | bit              | NOT NULL, default 1   | Lost/replaced cards deactivated |
| IssuedAt              | datetime2        | NOT NULL              |                                 |
| DeactivatedAt         | datetime2        | NULL                  |                                 |
| CreatedAt / UpdatedAt | datetime2        | NOT NULL              |                                 |

A student may have multiple cards over time; only one active per `CardUid`. The UID→Student lookup is the hot path — cache in Redis.

## 4.5 Events

| Column                | Type             | Constraints                | Description                     |
|-----------------------|------------------|----------------------------|---------------------------------|
| Id                    | uniqueidentifier | PK                         |                                 |
| SchoolId              | uniqueidentifier | FK→Schools, NOT NULL       |                                 |
| Name                  | nvarchar(200)    | NOT NULL                   | Event name                      |
| Description           | nvarchar(2000)   | NULL                       |                                 |
| Location              | nvarchar(300)    | NULL                       | Venue                           |
| StartAt               | datetime2        | NOT NULL                   | Event start (UTC)               |
| EndAt                 | datetime2        | NOT NULL                   | Event end (UTC)                 |
| AttendanceMode        | nvarchar(20)     | NOT NULL, default 'Single' | Single / TimelnOut              |
| GraceMinutes          | int              | NOT NULL, default 0        | Late threshold after StartAt    |
| RequireRegistration   | bit              | NOT NULL, default 0        | Restrict to associated students |
| Status                | nvarchar(20)     | NOT NULL, default 'Draft'  | Draft/Open/Closed/Cancelled     |
| OrganizerUserId       | uniqueidentifier | FK→Users, NULL             | Responsible organizer           |
| CreatedAt / UpdatedAt | datetime2        | NOT NULL                   |                                 |
| IsDeleted             | bit              | NOT NULL, default 0        |                                 |

## 4.6 EventSchedules (recurring events)

| Column                | Type             | Constraints         | Description             |
|-----------------------|------------------|---------------------|-------------------------|
| Id                    | uniqueidentifier | PK                  |                         |
| EventId               | uniqueidentifier | FK→Events, NOT NULL | Template event          |
| RecurrenceRule        | nvarchar(500)    | NULL                | iCal RRULE string       |
| OccurrenceStartAt     | datetime2        | NOT NULL            | Materialized occurrence |
| OccurrenceEndAt       | datetime2        | NOT NULL            |                         |
| IsCancelled           | bit              | NOT NULL, default 0 | Per-occurrence cancel   |
| CreatedAt / UpdatedAt | datetime2        | NOT NULL            |                         |

Recurring events are stored as a base `Event` + materialized `EventSchedules` occurrences (calendar view reads these). Attendance can reference either the event or a specific occurrence ( `OccurrenceId` nullable on `AttendanceRecords`).

## 4.7 StudentGroups & StudentGroupMembers

### StudentGroups

| Column                | Type             | Constraints          | Description                    |
|-----------------------|------------------|----------------------|--------------------------------|
| Id                    | uniqueidentifier | PK                   |                                |
| SchoolId              | uniqueidentifier | FK→Schools, NOT NULL |                                |
| Name                  | nvarchar(150)    | NOT NULL             | e.g. "BSIT 3A", "SSC Officers" |
| Type                  | nvarchar(30)     | NOT NULL             | Course/Section/Org/Custom      |
| CreatedAt / UpdatedAt | datetime2        | NOT NULL             |                                |

### StudentGroupMembers (junction)

| Column         | Type             | Constraints                      | Description   |
|----------------|------------------|----------------------------------|---------------|
| Id             | uniqueidentifier | PK                               |               |
| StudentGroupId | uniqueidentifier | FK→StudentGroups, NOT NULL       |               |
| StudentId      | uniqueidentifier | FK→Students, NOT NULL            |               |
|                |                  | UNIQUE(StudentGroupId,StudentId) | No duplicates |

## 4.8 EventGroups (associate events ↔ students/groups)

| Column         | Type             | Constraints                      | Description         |
|----------------|------------------|----------------------------------|---------------------|
| Id             | uniqueidentifier | PK                               |                     |
| EventId        | uniqueidentifier | FK→Events, NOT NULL              |                     |
| StudentGroupId | uniqueidentifier | FK→StudentGroups, NULL           | Whole group invited |
| StudentId      | uniqueidentifier | FK→Students, NULL                | Individual invited  |
|                |                  | CHECK (one of group/student set) | XOR                 |

Drives `RequireRegistration` validation and "expected attendees" denominators in reports.



## 4.9 AttendanceRecords (core table)

| Column                | Type             | Constraints                  | Description                    |
|-----------------------|------------------|------------------------------|--------------------------------|
| Id                    | uniqueidentifier | PK                           |                                |
| EventId               | uniqueidentifier | FK→Events, NOT NULL, INDEX   |                                |
| OccurrenceId          | uniqueidentifier | FK→EventSchedules, NULL      | If recurring                   |
| StudentId             | uniqueidentifier | FK→Students, NOT NULL, INDEX |                                |
| RfidCardId            | uniqueidentifier | FK→RfidCards, NULL           | Card used (null if manual)     |
| CheckInAt             | datetime2        | NULL                         | First tap (UTC)                |
| CheckOutAt            | datetime2        | NULL                         | Time-out tap (if TimeInOut)    |
| Status                | nvarchar(20)     | NOT NULL                     | Present/Late/Absent/Excused    |
| CaptureMethod         | nvarchar(20)     | NOT NULL, default 'Rfid'     | Rfid/Manual/Import             |
| DeviceId              | uniqueidentifier | FK→Devices, NULL             | Capturing device               |
| DeviceTapId           | nvarchar(100)    | NULL                         | Client-generated id for dedupe |
| Notes                 | nvarchar(500)    | NULL                         |                                |
| RecordedByUserId      | uniqueidentifier | FK→Users, NULL               | Manual entry author            |
| CreatedAt / UpdatedAt | datetime2        | NOT NULL                     |                                |

**Constraints/Indexes:** - `UNIQUE(EventId, StudentId, OccurrenceId)` — one attendance per student per event/occurrence (time-in/out updates the same row). - `UNIQUE(DeviceId, DeviceTapId)` (filtered, where DeviceTapId NOT NULL) — idempotent offline sync. - `IX_Attendance_EventId_Status` — report aggregation.

## 4.10 Devices (RFID readers / kiosks)

| Column                | Type             | Constraints          | Description           |
|-----------------------|------------------|----------------------|-----------------------|
| Id                    | uniqueidentifier | PK                   |                       |
| SchoolId              | uniqueidentifier | FK→Schools, NOT NULL |                       |
| Name                  | nvarchar(100)    | NOT NULL             | "Gym Kiosk 1"         |
| DeviceType            | nvarchar(30)     | NOT NULL             | Mobile/Kiosk/Handheld |
| ReaderModel           | nvarchar(100)    | NULL                 | Hardware model        |
| ApiKey                | nvarchar(256)    | NULL, UNIQUE         | For kiosk auth        |
| LastSeenAt            | datetime2        | NULL                 | Heartbeat             |
| IsActive              | bit              | NOT NULL, default 1  |                       |
| CreatedAt / UpdatedAt | datetime2        | NOT NULL             |                       |

## 4.11 RBAC tables

Users

| Column                | Type             | Constraints          | Description            |
|-----------------------|------------------|----------------------|------------------------|
| Id                    | uniqueidentifier | PK                   |                        |
| SchoolId              | uniqueidentifier | FK→Schools, NOT NULL |                        |
| Email                 | nvarchar(256)    | NOT NULL, UNIQUE     | Login                  |
| PasswordHash          | nvarchar(max)    | NOT NULL             | ASP.NET Identity hash  |
| FullName              | nvarchar(200)    | NOT NULL             |                        |
| Phone                 | nvarchar(30)     | NULL                 |                        |
| IsActive              | bit              | NOT NULL, default 1  |                        |
| LastLoginAt           | datetime2        | NULL                 |                        |
| RefreshTokenHash      | nvarchar(max)    | NULL                 | Rotating refresh token |
| CreatedAt / UpdatedAt | datetime2        | NOT NULL             |                        |

**Roles** — Id, Name (UNIQUE: SuperAdmin/SchoolAdmin/Organizer/Viewer), Description, IsSystem (bit).  
**Permissions** — Id, Code (UNIQUE, e.g. students.read, events.write, attendance.capture, reports.export), Description.  
**UserRoles** — junction UserId × RoleId (UNIQUE). **RolePermissions** — junction RoleId × PermissionId (UNIQUE).

## 4.12 SIS import tables

SisImportBatches

| Column                 | Type             | Constraints          | Description                      |
|------------------------|------------------|----------------------|----------------------------------|
| Id                     | uniqueidentifier | PK                   |                                  |
| SchoolId               | uniqueidentifier | FK→Schools, NOT NULL |                                  |
| Source                 | nvarchar(20)     | NOT NULL             | Csv/DbLink/Api                   |
| FileName               | nvarchar(300)    | NULL                 | Original upload                  |
| Status                 | nvarchar(20)     | NOT NULL             | Pending/Running/Completed/Failed |
| TotalRows              | int              | NOT NULL, default 0  |                                  |
| InsertedRows           | int              | NOT NULL, default 0  |                                  |
| UpdatedRows            | int              | NOT NULL, default 0  |                                  |
| FailedRows             | int              | NOT NULL, default 0  |                                  |
| StartedAt / FinishedAt | datetime2        | NULL                 |                                  |
| RunByUserId            | uniqueidentifier | FK→Users, NULL       |                                  |

**SisImportRows** — Id, BatchId (FK), RowNumber, RawData (nvarchar(max), JSON of source row), Result (Inserted/Updated/Failed/Skipped), ErrorMessage, StudentId (FK, nullable).

## 4.13 AuditLogs & SystemSettings

**AuditLogs** — Id, UserId (FK, nullable), Action (e.g. Event.Updated), EntityType, EntityId, Changes (nvarchar(max) JSON before/after), IPAddress, CreatedAt. **SystemSettings** — Id, SchoolId (FK, nullable for global), Key (UNIQUE per school), Value (nvarchar(max)), DataType, Description. Stores RFID reader defaults, grace minutes, report branding, retention policy, etc.

## 5. Backend Modules (.NET)

Each module = a feature folder in **EAMS.Application** (DTOs, validators, service interface + impl) exposed by a controller in **EAMS.Api**.

| Module                  | Responsibility                                                 | Key services                                   |
|-------------------------|----------------------------------------------------------------|------------------------------------------------|
| <b>Auth</b>             | Login, refresh, password reset, token issuance                 | IAuthService, IJwtTokenService                 |
| <b>Users &amp; RBAC</b> | CRUD users, assign roles, manage roles/permissions             | IUserService, IRoleService, IPermissionService |
| <b>Students</b>         | CRUD, search, filter, photo, card assignment                   | IStudentService, IRfidCardService              |
| <b>Groups</b>           | Student groups & membership                                    | IStudentGroupService                           |
| <b>Events</b>           | CRUD events, status transitions, associate attendees           | IEventService, IEventGroupService              |
| <b>Schedules</b>        | Recurrence expansion, calendar feed                            | IScheduleService (RRULE engine)                |
| <b>Attendance</b>       | Tap capture, manual entry, dedupe, status calc, live broadcast | IAttendanceService, AttendanceHub (SignalR)    |
| <b>Devices</b>          | Register readers/kiosks, API keys, heartbeat                   | IDeviceService                                 |
| <b>Reports</b>          | Aggregations + PDF/CSV export                                  | IReportService, IExportService                 |
| <b>SIS Import</b>       | CSV/DB/API ingestion, mapping, batch tracking                  | ISisImportService, Hangfire jobs               |
| <b>Settings</b>         | System/school configuration                                    | ISettingsService                               |
| <b>Audit</b>            | Change logging                                                 | IAuditService (interceptor-based)              |

**Attendance status logic** (computed at capture and at event close): - Tap before **StartAt** + **GraceMinutes** → **Present**. - Tap after grace → **Late**. - No tap by **EndAt** for an expected attendee → **Absent** (batch job at event close). - **Excused** set manually by an organizer/admin.

## 6. API Specification

**Base URL:** /api/v1 · **Format:** JSON · **Auth:** Authorization: Bearer <JWT> · **Errors:** RFC 7807 ProblemDetails. List endpoints support ?page=&pageSize=&search=&sort=.

## 6.1 Auth

| Method | Path                  | Body                 | Permission | Description                                              |
|--------|-----------------------|----------------------|------------|----------------------------------------------------------|
| POST   | /auth/login           | {email, password}    | anon       | Returns {accessToken, refreshToken, user, permissions[]} |
| POST   | /auth/refresh         | {refreshToken}       | anon       | Rotates tokens                                           |
| POST   | /auth/logout          | {refreshToken}       | auth       | Revokes refresh token                                    |
| POST   | /auth/forgot-password | {email}              | anon       | Sends reset link                                         |
| POST   | /auth/reset-password  | {token, newPassword} | anon       |                                                          |
| GET    | /auth/me              | —                    | auth       | Current user + permissions                               |

## 6.2 Students

| Method | Path                          | Permission         | Description                                                |
|--------|-------------------------------|--------------------|------------------------------------------------------------|
| GET    | /students                     | students.read      | Paged list; filters: course , yearLevel , groupId , status |
| GET    | /students/{id}                | students.read      | Detail incl. cards & group memberships                     |
| POST   | /students                     | students.write     | Create                                                     |
| PUT    | /students/{id}                | students.write     | Update                                                     |
| DELETE | /students/{id}                | students.write     | Soft delete                                                |
| POST   | /students/{id}/cards          | students.write     | Assign RFID card {cardUid, label}                          |
| DELETE | /students/{id}/cards/{cardId} | students.write     | Deactivate card                                            |
| GET    | /students/by-card/{cardUid}   | attendance.capture | Resolve UID→student (mobile)                               |

## 6.3 Events & Schedules

| Method | Path                       | Permission   | Description                            |
|--------|----------------------------|--------------|----------------------------------------|
| GET    | /events                    | events.read  | Paged; filters: status , from , to     |
| GET    | /events/{id}               | events.read  | Detail + associated groups/students    |
| POST   | /events                    | events.write | Create                                 |
| PUT    | /events/{id}               | events.write | Update                                 |
| PATCH  | /events/{id}/status        | events.write | Open/Close/Cancel                      |
| DELETE | /events/{id}               | events.write | Soft delete                            |
| POST   | /events/{id}/attendees     | events.write | Associate {studentIds[], groupId[]}    |
| GET    | /events/{id}/roster        | events.read  | Expected vs present roster             |
| GET    | /events/calendar?from=&to= | events.read  | Calendar occurrences (incl. recurring) |
| POST   | /events/{id}/schedules     | events.write | Add recurrence {rrule, start, end}     |

## 6.4 Attendance

| Method | Path                       | Permission         | Description                                                                                                                                      |
|--------|----------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| POST   | /attendance/tap            | attendance.capture | <b>Core.</b> {eventId, occurrenceId?, cardUid, deviceId, deviceTapId, tappedAt} → resolves student, validates window, upserts record, broadcasts |
| POST   | /attendance/tap/batch      | attendance.capture | Offline sync — array of taps; idempotent by deviceTapId                                                                                          |
| POST   | /attendance/manual         | attendance.write   | {eventId, studentId, status, checkInAt?, notes}                                                                                                  |
| PUT    | /attendance/{id}           | attendance.write   | Edit status/notes (audited)                                                                                                                      |
| GET    | /attendance                | attendance.read    | Filters: eventId , studentId , status , date range                                                                                               |
| GET    | /attendance/live/{eventId} | attendance.read    | Snapshot for dashboard (SignalR pushes deltas)                                                                                                   |

**SignalR hub** /hubs/attendance — group per eventId ; server emits attendanceUpdated events {eventId, studentId, status, checkInAt, presentCount, expectedCount} .

## 6.5 Users & RBAC

| Method   | Path                                      | Permission               |
|----------|-------------------------------------------|--------------------------|
| GET/POST | /users , /users/{id} (GET/PUT/DELETE)     | users.read / users.write |
| POST     | /users/{id}/roles {roleIds[]}             | users.write              |
| GET/POST | /roles , /roles/{id}                      | roles.read / roles.write |
| POST     | /roles/{id}/permissions {permissionIds[]} | roles.write              |
| GET      | /permissions                              | roles.read               |

## 6.6 Devices

| Method   | Path                         | Permission                   | Description     |
|----------|------------------------------|------------------------------|-----------------|
| GET/POST | /devices                     | devices.read / devices.write | List / register |
| POST     | /devices/{id}/regenerate-key | devices.write                | New API key     |
| POST     | /devices/{id}/heartbeat      | device key                   | Liveness        |

## 6.7 Reports

| Method | Path                                       | Permission     | Description                                                                         |
|--------|--------------------------------------------|----------------|-------------------------------------------------------------------------------------|
| GET    | /reports/event/{eventId}/summary           | reports.read   | Present/Late/Absent/Excused counts + rate                                           |
| GET    | /reports/student/{studentId}/history       | reports.read   | Attendance across events                                                            |
| GET    | /reports/absentees?eventId=                | reports.read   | Expected but absent                                                                 |
| GET    | /reports/attendance?<br>from=&to=&groupId= | reports.read   | Cross-event analytics                                                               |
| POST   | /reports/export                            | reports.export | {reportType, params, format: Pdf\ Csv} → file (sync) or job id (async via Hangfire) |

## 6.8 SIS Import

| Method | Path                                         | Permission | Description                                            |
|--------|----------------------------------------------|------------|--------------------------------------------------------|
| POST   | /sis/import/upload                           | sis.import | Multipart CSV → creates batch, returns preview/mapping |
| POST   | /sis/import/{batchId}/run                    | sis.import | Enqueue Hangfire job with field mapping                |
| GET    | /sis/import/{batchId}                        | sis.import | Batch status + counts                                  |
| GET    | /sis/import/{batchId}/rows?<br>result=Failed | sis.import | Row-level results/errors                               |

## 6.9 Settings

| Method  | Path      | Permission                     |
|---------|-----------|--------------------------------|
| GET/PUT | /settings | settings.read / settings.write |

## 7. Frontend — Web Admin (React)

**Purpose:** administration, configuration, monitoring, and reporting. Not used for tapping (that's mobile/kiosk).

### 7.1 Routes / Pages

| Route              | Page                                      | Guard (permission) |
|--------------------|-------------------------------------------|--------------------|
| /login             | Login                                     | anon               |
| /dashboard         | KPIs, today's events, live counts         | attendance.read    |
| /students          | List + search/filter                      | students.read      |
| /students/:id      | Profile, cards, group memberships         | students.read      |
| /groups            | Student groups & members                  | students.read      |
| /events            | List + calendar view                      | events.read        |
| /events/:id        | Detail, attendees, <b>live attendance</b> | events.read        |
| /events/:id/roster | Expected vs present                       | events.read        |
| /reports           | Report builder + export                   | reports.read       |
| /users             | Users, roles assignment                   | users.read         |
| /roles             | Roles & permission matrix                 | roles.read         |
| /devices           | Readers/kiosks, API keys                  | devices.read       |
| /sis-import        | Upload, map, run, monitor batches         | sis.import         |
| /settings          | School + system config                    | settings.read      |

### 7.2 Key components & patterns

- `<PermissionGuard permission="...">` wraps routes/buttons; hides/disables based on `me.permissions`.
- `<DataTable>` — reusable server-paginated MUI table (sort/filter/search bound to query params).
- **RTK Query slices** per module with tag-based cache invalidation (e.g. mutating an event invalidates `Events` + `Roster`).
- **Live dashboard** — `useSignalR('/hubs/attendance')` subscribes to the open event; updates counters and roster rows in place.
- **Calendar** — FullCalendar/MUI rendering `/events/calendar` occurrences.
- **Forms** — React Hook Form + Zod schemas mirroring backend FluentValidation.

## 8. Mobile App — RFID Capture (React Native)

---

**Purpose:** the tapping device — handheld scanner, organizer's phone, or fixed kiosk. Android-first (most RFID handhelds run Android).

### 8.1 Screens

| Screen              | Function                                                                                                                                  |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Login</b>        | Auth as organizer, or kiosk mode via device API key                                                                                       |
| <b>Event Picker</b> | Choose the active/open event (or scheduled occurrence)                                                                                    |
| <b>Scan Screen</b>  | Big status display; listens for taps; shows last student (name, photo, Present/Late), running present count, online/offline + queue badge |
| <b>Roster</b>       | Searchable expected list; manual check-in fallback; mark excused                                                                          |
| <b>Sync / Queue</b> | Pending offline taps, retry, last sync time                                                                                               |
| <b>Settings</b>     | Reader pairing (BLE), tap mode (single / time-in-out), sound/vibration feedback                                                           |

### 8.2 Offline-first sync engine

1. Tap captured → resolve UID locally if student cache present, else queue raw UID.
2. Write to `expo-sqlite` queue: `{localId(uuid), eventId, occurrenceId, cardUid, deviceTapId, tappedAt, synced:0}`.
3. Connectivity watcher flushes queue via `POST /attendance/tap/batch`; server dedupes by `(deviceId, deviceTapId)`.
4. On success mark `synced:1`; on conflict (already present) treat as success.
5. Periodic pull of the event's student roster for offline UID→name resolution and manual fallback.

`deviceTapId` is a client-generated UUID — the keystone of idempotency. The same tap retried any number of times produces exactly one record.

---

## 9. RFID Integration Design

---

**Reader interface options (adapter pattern in `mobile-rfid/src/rfid/`):**



| Mode                              | How it works                                                                | Best for                                   |
|-----------------------------------|-----------------------------------------------------------------------------|--------------------------------------------|
| <b>Keyboard-wedge</b>             | Reader emulates a keyboard; UID arrives as text + Enter into a hidden input | Cheapest; USB/BT handhelds; near-zero code |
| <b>BLE SDK</b>                    | Pair reader over Bluetooth LE; subscribe to tag-read characteristic         | Mobile organizers, untethered              |
| <b>Vendor SDK / native module</b> | Integrated Android handheld (e.g. Chainway/Zebra) via native module         | Dedicated kiosks/handhelds                 |

All adapters implement a common contract:

```
interface RfidReader {
  start(onTag: (uid: string) => void): Promise<void>;
  stop(): Promise<void>;
  status(): ReaderStatus;
}
```

The Scan screen is reader-agnostic — swapping hardware only changes the adapter. UID normalization (uppercase hex, strip separators) happens once in the adapter so the API always receives canonical UIDs matching `RfidCards.CardUid`.

**Note (per proposal):** RFID hardware is procured separately. Recommend testing 1–2 reader models during Discovery to lock the adapter.

## 10. SIS Data Migration

**Pipeline:** Source → Staging ( `SisImportBatches` / `SisImportRows` ) → Validation → Upsert into `Students` .

1. **Source connectors:** CSV upload (MVP), direct DB read (read-only view/credentials), or SIS REST API pull.
2. **Mapping UI:** user maps source columns → `Students` fields ( `StudentNumber` , names, `Course` , `YearLevel` , `Section` , `Email` ). Mapping saved per school in `SystemSettings` for repeat runs.
3. **Upsert key:** `SisExternalId` (preferred) or `StudentNumber` within school. Existing → update + `LastSyncedAt` ; new → insert.
4. **Idempotent & re-runnable:** each batch tracked; failed rows carry `ErrorMessage` and are individually retryable.
5. **Scheduling:** ongoing sync runs as a recurring Hangfire job (for DB/API sources).

## 11. Authentication, Authorization & RBAC

- **AuthN:** JWT access token (~15 min) + rotating refresh token (~7 days, hashed at rest). ASP.NET Identity for password hashing.

- **AuthZ:** permission-based, not just role-based. Permissions are fine-grained codes ( `students.write` , `attendance.capture` ...); roles are bundles. The JWT carries role claims; the API resolves effective permissions and enforces via a policy/ `[HasPermission("events.write")]` attribute. Frontend mirrors permissions for UI gating only — server is authoritative.
- **Default roles:** SuperAdmin (all), SchoolAdmin (everything within school), Organizer (events + attendance + read students/reports), Viewer (read + export reports).
- **Device auth:** kiosks authenticate with a long-lived device API key scoped to `attendance.capture` only.
- **Multi-tenant guard:** every query is filtered by `SchoolId` from the user's claims (EF Core global query filter) to prevent cross-school data access.

## 12. Reporting Module

| Report                            | Contents                                                                    | Export   |
|-----------------------------------|-----------------------------------------------------------------------------|----------|
| <b>Event Attendance Summary</b>   | Present / Late / Absent / Excused counts, attendance rate, timeline of taps | PDF, CSV |
| <b>Student Attendance History</b> | All events for a student, status per event                                  | PDF, CSV |
| <b>Absentee Report</b>            | Expected attendees with no tap                                              | PDF, CSV |
| <b>Group/Course Analytics</b>     | Attendance rate by group/course/year over a date range                      | PDF, CSV |
| <b>Per-Event Roster</b>           | Expected vs actual with check-in times                                      | PDF, CSV |

- Aggregations computed in SQL (indexed on `EventId` , `Status` ).
- PDF via **QuestPDF** (branded with school logo from `Schools.LogoUrl` ); CSV via **CsvHelper**.
- Large exports run as Hangfire jobs; the API returns a job id and a download link when ready.

## 13. Deployment

Both options share one Dockerized codebase; only hosting differs (per proposal §6).

### Option A — SaaS (managed by vendor)

- API + Worker + DB (managed SQL) + Redis on cloud (Azure App Service / Container Apps).
- Web admin as static SPA on CDN; mobile distributed via store/MDM.
- Per-school tenant isolation by `SchoolId` ; automatic updates, backups, monitoring handled by vendor.

### Option B — On-Premise (school-hosted)

- `docker-compose` bundle: `api` , `worker` , `db` (SQL Server), `redis` , `web` (nginx serving SPA).
- One-time license; school provides server, handles backups/network; full source code delivered.
- Installation & configuration guide + deployment plan included.

| Concern       | SaaS            | On-Premise                          |
|---------------|-----------------|-------------------------------------|
| Hosting       | Vendor cloud    | School servers                      |
| Updates       | Automatic       | School-applied (with guidance)      |
| Data location | Vendor cloud    | School network (data sovereignty)   |
| Cost model    | Subscription    | One-time license + optional support |
| RFID readers  | Client-provided | Client-provided                     |

## 14. Cross-Cutting Concerns

- **Validation:** FluentValidation on all DTOs; client Zod schemas mirror them.
- **Error handling:** global exception middleware → ProblemDetails; correlation id per request.
- **Logging/observability:** Serilog structured logs; request logging; SignalR + import metrics.
- **Security:** HTTPS only, HSTS; rate limiting on `/auth` and `/attendance/tap`; parameterized EF queries; secrets in env/Key Vault; audit logging of all writes; CORS locked to known origins.
- **Performance:** Redis cache for UID→student map; paginated lists; DB indexes on hot paths; batch tap endpoint to cut round-trips.
- **Data retention:** configurable purge of old `SisImportRows` and `AuditLogs` via Hangfire (policy in `SystemSettings`).
- **Testing:** xUnit unit + integration (Testcontainers for DB) on backend; Vitest + React Testing Library on web; Jest + RNTL on mobile; idempotency/dedup tests on attendance.
- **API versioning:** URL-based `/api/v1`.

## 15. Implementation Roadmap

Mapped to the proposal's phases.

| Phase                                  | Duration         | Deliverables                                                                                                                                           |
|----------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1 — Discovery &amp; Planning</b>    | 1–2 wks          | Finalize requirements, SIS data mapping, pick RFID reader model(s), confirm DB provider & deployment option, lock API contract (OpenAPI)               |
| <b>2 — Web System Development</b>      | 8–10 wks         | DB schema + EF migrations; Auth/RBAC; Students/Groups; Events/Schedules; Attendance + SignalR; Devices; Settings; Web admin SPA for all above; Swagger |
| <b>3 — Mobile / RFID</b>               | (within Phase 2) | RN app: login, event picker, scan screen, offline queue/sync, roster fallback; RFID adapters; device registration                                      |
| <b>4 — SIS Migration &amp; Testing</b> | 1–2 wks          | Import pipeline (CSV → DB/API); mapping UI; UAT; performance & security testing; idempotency tests                                                     |
| <b>5 — Deployment &amp; Training</b>   | 1–2 wks          | Dockerized deploy (SaaS or on-prem); training for admins/organizers; documentation handover (user manuals, technical docs, install guide)              |
| <b>6 — Post-Deployment Support</b>     | Ongoing          | Bug fixes, monitoring, enhancements per support agreement                                                                                              |

## Suggested build order (dependency-driven)

1. Solution scaffolding, DB schema, EF migrations, seed roles/permissions.
  2. Auth + RBAC (unblocks everything).
  3. Students + RFID cards + SIS CSV import (need data to test taps).
  4. Events + Schedules + attendee association.
  5. Attendance capture (API) → mobile scan → offline sync → SignalR live dashboard.
  6. Reports + exports.
  7. Devices, Settings, Audit polish.
  8. Hardening, tests, Dockerization, deployment.
- 

*End of plan. This document defines the contract — the API spec (§6) and schema (§4) are the source of truth for parallel frontend/mobile/backend work.*